C++ COMPLETE REFERENCE LIBRARY • MODULE 20

20 TYPE CONVERSION AND CASTING — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 implicit and user defined conversions
3. **Theory Chapter:** 02 the four cpp named casts
4. **Theory Chapter:** 03 dynamic cast and rtti rules
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

20 — Type Conversion and C++ Named Casts

This module covers implicit conversions, user-defined conversions (converting constructors, `explicit`, and `operator int()`), and the four C++ named casts (`static_cast`, `dynamic_cast`, `const_cast`, and `reinterpret_cast`).

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_implicit_and_user_defined_conversions.md` (./01_THEORY/01_implicit_and_user_defined_conversions.md)
  - `02_the_four_cpp_named_casts.md` (./01_THEORY/02_the_four_cpp_named_casts.md)
  - `03_dynamic_cast_and_rtti_rules.md` (./01_THEORY/03_dynamic_cast_and_rtti_rules.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_class_type_conversions.cpp` (./02_PROGRAMS/01_class_type_conversions.cpp) (Category A)
  - `02_static_and_dynamic_cast.cpp` (./02_PROGRAMS/02_static_and_dynamic_cast.cpp) (Category A)

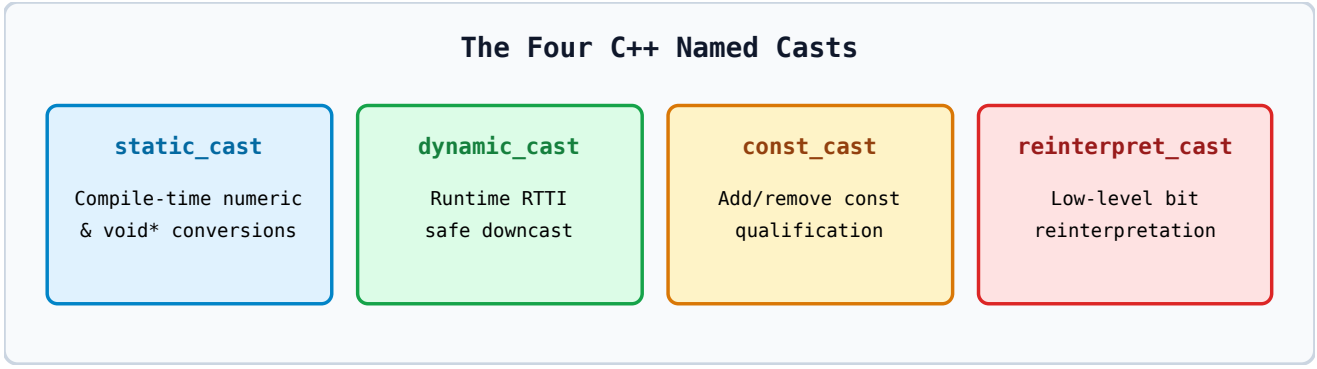
  - `03_explicit_constructor_compile_error.cpp` (./02_PROGRAMS/03_explicit_constructor_compile_error.cpp) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_cpp_named_casts.svg` (./05_DIAGRAMS/diagram_01_cpp_named_casts.svg)

01 — Implicit Promotions and User-Defined Type Conversions

1. Three Classroom Type-Conversion Scenarios

1. **Basic Type to Class Type (`int → Class`)**: Handled by a **single-argument constructor** (unless marked `explicit`).
2. **Class Type to Basic Type (`Class → int`)**: Handled by an overloaded **conversion operator** inside the class: `operator int() const { return total_minutes; }` (note: has no return type written before `operator`, but returns `int`).
3. **Class A to Class B**: Handled either by a constructor in `Class B` taking `const A&`, or a conversion operator `operator B() const` inside `Class A`.

02 — The Four C++ Named Casts (`static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast`)



C++ Named Casts

Named Cast	Primary Purpose	Checked When?
`static_cast<T>(expr)`	Well-defined numeric conversions (`double` to `int`), `void*` to `T*`, related class hierarchy casts	Compile time
`dynamic_cast<T>(expr)`	Safe runtime downcasting (`Base*` to `Derived*`) in polymorphic classes (requires `virtual` function in `Base`)	**Runtime (RTTI)**
`const_cast<T>(expr)`	Adding or removing `const` / `volatile` qualification	Compile time
`reinterpret_cast<T>(expr)`	Low-level bit/address reinterpretation between unrelated pointer types	Compile time

03 — `dynamic_cast` Failure Behavior: Pointer vs. Reference

When a downcast `dynamic_cast<Derived*>(base_ptr)` fails at runtime (because `base_ptr` does not actually point to a `Derived` object):

- **Pointer** `dynamic_cast<Derived*>(base_ptr)`: Returns `nullptr`.
- **Reference** `dynamic_cast<Derived&>(base_ref)`: Throws `std::bad_cast` (`#include <typeinfo>`) because there is no such thing as a null reference!

Visual Concept & Memory Diagrams (20 TYPE CONVERSION AND CASTING)

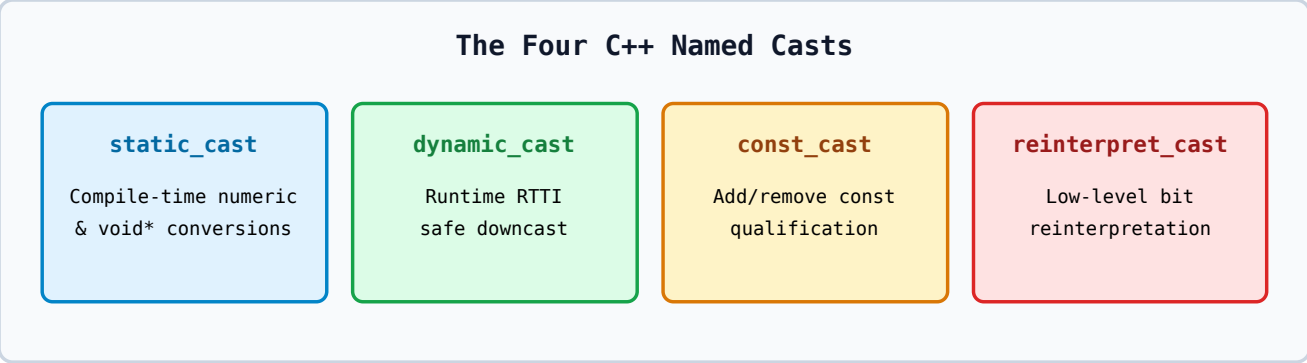


Diagram: diagram 01 cpp named casts

Annotated C++ Programs (20 TYPE CONVERSION AND CASTING)

Program: `01_class_type_conversions.cpp`

C++ SOURCE FILE • 01_class_type_conversions.cpp

```cpp
// Program Title: Basic-to-Class and Class-to-Basic Type Conversions
// Topic: Type Conversion and Casting — User-Defined Conversions
// Purpose: Demonstrate converting int minutes into a time_duration object and converting
back to int.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_implicit_and_user_defined_conversions.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class time_duration {
private:
    int hours;
    int mins;

public:
    // 1. Basic to Class conversion constructor
    time_duration(int total_mins) {
        hours = total_mins / 60;
        mins = total_mins % 60;
    }

    // 2. Class to Basic conversion operator
    operator int() const {
        return (hours * 60) + mins;
    }

    void display() const {
        cout << hours << " hrs " << mins << " mins" << endl;
    }
};

int main() {
    time_duration t = 135; // Implicit int → time_duration conversion
    t.display();

    int total = t;         // Implicit time_duration → int conversion
    cout << "Total minutes: " << total << endl;
    return 0;
}

/*
Expected Deterministic Output:
2 hrs 15 mins
Total minutes: 135
*/
```

Program: `02_static_and_dynamic_cast.cpp`

C++ SOURCE FILE • 02_static_and_dynamic_cast.cpp

```cpp
// Program Title: Safe Numeric static_cast and Runtime Polymorphic dynamic_cast
// Topic: Type Conversion and Casting — Named Casts
// Purpose: Demonstrate static_cast<double> and checking dynamic_cast<student*> against
nullptr.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_the_four_cpp_named_casts.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

class person {
public:
    virtual ~person() = default;
};

class student : public person {
public:
    void study() const { cout << "Student is studying." << endl; }
};

int main() {
    int a = 7, b = 2;
    cout << "static_cast division: " << static_cast<double>(a) / b << endl;

    student s;
    person* base_ptr = &s;

    student* derived_ptr = dynamic_cast<student*>(base_ptr);
    if (derived_ptr != nullptr) {
        derived_ptr->study();
    }

    return 0;
}

/*
Expected Deterministic Output:
static_cast division: 3.5
Student is studying.
*/
```

Program: `03_explicit_constructor_compile_error.cpp`

C++ SOURCE FILE • 03_explicit_constructor_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Implicit Copy-Initialization Blocked by explicit Constructor
// Topic: Type Conversion and Casting — explicit Specifier
// Purpose: Show why 'person p = 20;' fails when 'explicit person(int a)' is used.
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/01_implicit_and_user_defined_conversions.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

class person {
private:
    int age;

public:
    explicit person(int a) : age(a) {}
};

int main() {
    person p = 20; // COMPILE ERROR: conversion from 'int' to non-scalar type 'person'
requested
    return 0;
}

/*
Why Compilation Fails:
Marking the constructor 'explicit' prevents implicit conversions in copy-initialization
('person p = 20;').
Use direct initialization 'person p(20);' or 'person p{20};'.
*/
```

Conceptual Questions — Module 20: Type Conversion and Casting

Q1. Compare `static_cast` and `dynamic_cast`. Why does `dynamic_cast` require the base class to have at least one `virtual` function? **Q2.** What happens if you use `const_cast` to remove `const` from a pointer to a variable that was originally declared `const int x = 10;` and then write to it?

Debugging Questions — Module 20

Q4. Why does `int operator int() const` fail to compile inside a class?

Predict the Output Questions — Module 20

Q3. What does `dynamic_cast<Derived*>(base_ptr)` return if `base_ptr` actually points to a plain `Base` object?

Programming Questions — Module 20

Q5. Write a C++23 classroom program converting an object of `class rupee` into `class dollar` using a conversion constructor.

Detailed Solutions — Conceptual Questions (Module 20)

- **Q1 Solution**: `static_cast` performs compile-time conversion without runtime type checking; `dynamic_cast` inspects the object's RTTI (stored via the class's `vtable`, which is why a polymorphic `virtual` function is required) at runtime.
- **Q2 Solution**: Modifying an object that was originally declared `const` through a `const_cast` pointer is **Undefined Behavior** (`[dcl.type.cv]`).

Detailed Solutions — Predict the Output & Debugging (Module 20)

- **Q3 Solution**: It returns `nullptr`.
- **Q4 Solution**: Conversion operators (`operator int() const`) do not allow an explicit return type before `operator` because the return type is already specified by the target type name (`int`).

Detailed Solutions — Programming Questions (Module 20)

CPP

```cpp
#include <iostream>
using namespace std;
class rupee {
    int inr;
public:
    rupee(int r) : inr(r) {}
    int get_inr() const { return inr; }
};
class dollar {
    int usd;
public:
    dollar(const rupee& r) : usd(r.get_inr() / 80) {}
    void show() const { cout << "USD: " << usd << endl; }
};
int main() {
    rupee r(400);
    dollar d = r;
    d.show();
    return 0;
}
```